



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorio de docencia

Laboratorios de computación salas A y B

Profesor: Elba Karen Sáenz García

Asignatura: Estructura de Datos y Algoritmos I

Grupo: 4

No de Práctica(s): 2-3

Integrante(s): - Ramírez Jiménez Aram

- Romero Pizano Christian Gustavo

- Silverio Martínez Andrés

No. de Lista o Brigada: _____

Semestre: 2022 - 2

Fecha de entrega: 11 de Marzo de 2022

CALIFICACIÓN: _____

Práctica 2

Aplicaciones de apuntadores

Objetivo

Utilizar apuntadores en Lenguaje C para acceder a localidades de memoria tanto de datos primitivos como arreglos.

Desarrollo

Actividad 1

Completar el siguiente código:

```
#include <stdio.h>

main(){
double a,*ap;
a=7.5;
ap=&a;
printf("Direccion asignada a la variable a % \n", );
printf("Direccion asignada a la variable ap % \n", );
printf(" Valor asignado a la variable a % \n", );
printf(" Valor asignado a la variable ap o dirección a la que apunta % \n", );
/*Escribir abajo la línea que modifique el valor dela variable a mediante el
apuntador ap y se muestre el nuevo valor en la salida estándar*/
return 0;
}
```

```
1  #include <stdio.h>
2
3  int main(){
4      double a, *ap;
5      a=7.5;
6      ap=&a;
7      printf("Direccion asignada a la variable a: %p \n", &a);
8      printf("Direccion asignada a la variable ap: %p \n", &ap);
9      printf("Valor asignado a la variable 'a': %.3lf \n", a);
10     printf("Valor asignado a la variable ap o direccion a la que apunta: %p \n", ap);
11     *ap=2;
12     printf("Nuevo valor de la variable: %.3lf", *ap);
13
14
15     return 0;
16 }
```

```
C:\Users\andik\Desktop\Ejercicio 1.exe
Direccion asignada a la variable a: 000000000062FE18
Direccion asignada a la variable ap: 000000000062FE10
Valor asignado a la variable 'a': 7.500
Valor asignado a la variable ap o direccion a la que apunta: 000000000062FE18
Nuevo valor de la variable: 2.000
-----
Process exited after 0.09418 seconds with return value 0
Presione una tecla para continuar . . .
```

Actividad 2

Completa el siguiente código:

```
#include <stdio.h>

int main () {
    short arr[5], *apArr, i;
    printf("Dirección del primer elemento del arreglo arr: % \n", );
    printf("Dirección asignada a arr: % \n",);
    apArr = &arr[0];
    printf("Dirección asignada a la variable apuntador
    apArr: % \n",);
    printf("Dirección almacenada en variable apuntador
    apArr: % \n", );
    /*Colocar código para llenar el arreglo arr utilizando el apuntador variable apArr
    */

    /* Colocar código para incrementar en 2 el arreglo arr utilizando el nombre del
    arreglo como apuntador constante */

    /* Colocar código para imprimir los elementos del arreglo utilizando notación
    apuntador*/
    return 0;
}
```

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  int main(){
4      short arr[5], *apArr, i;
5      printf("Direccion del primer elemento del arreglo arr: %p \n", &arr[0]);
6      printf("Direccion asignada a arr: %p \n", &arr);
7
8      apArr=&arr[0];
9
10     printf("Direccion asignada a la variable apuntador apArr: %p \n",&apArr);
11     printf("Direccion almacenada en la variable apuntador apArr: %p \n",apArr);
12
13     for(i=0;i<5;i++){
14         *(apArr+i)= i+1; //Llena el arreglo a través del apuntador
15
16         for(i=0;i<5;i++){
17             *(apArr+i)+= 2; //Se le suma 2 a cada elemento del arreglo
18
19             for(i=0;i<5;i++){
20                 printf("\nElemento %d: %d*",i+1, *(apArr+i)); //Imprime los elementos del arreglo
21             }
22         }
23     }
24     return 0;
25 }

```

```

C:\Users\andik\Desktop\Ejercicio 2.exe
Direccion del primer elemento del arreglo arr: 000000000062FE10
Direccion asignada a arr: 000000000062FE10
Direccion asignada a la variable apuntador apArr: 000000000062FE08
Direccion almacenada en la variable apuntador apArr: 000000000062FE10

Elemento 1: 3*
Elemento 2: 4*
Elemento 3: 5*
Elemento 4: 6*
Elemento 5: 7*
-----
Process exited after 0.1059 seconds with return value 0
Presione una tecla para continuar . . .

```

Actividad 3

Utilizando los apuntadores variables a enteros ap, bp llenar dos arreglos unidimensionales estáticos A y B de tamaño 10 y después con un apuntador a entero cp llenar un arreglo C de tamaño 20 tomando primero un elemento de A y luego uno de B de forma sucesiva (intercalar un elemento de A y uno de B en C) .Se deben tener las funciones llena(), llenaC() e imprime() de manera que el programa y la función main() tenga la siguiente estructura:

```
#include<stdio.h>

#define n 10
#define m 20

/*Agregar aquí los prototipos de las funciones*/

Int main(){
int A[n] ,B[n] ,C[m],*ap,*bp,*cp;
ap= ;
    llena(ap);
bp= ; // asignar valor correspondiente a b
    llena(bp);
cp = llenaC(ap,bp,cp);
    imprime(ap,n);
    imprime(bp,n);
    imprime(cp,m);
return 0;
}

/* Agregar definición de las funciones*/

OJO: aquí solo deben definirse 3 funciones llena() , imprime() y llenaC()
```

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #define n 10
4  #define m 20
5
6  void llena(int *p);
7  void llenaC(int *a, int *b, int *c);
8  void imprime(int *p, int ne);
9
10 int main(){
11     int A[n], B[n], C[m];
12     int *ap, *bp, *cp;
13
14     ap = A;
15     bp = B;
16     cp = C;
17
18     llena(ap);
19     llena(bp);
20     llenaC(ap, bp, cp);
21     printf("Arreglo 1:\n\n");
22     imprime(ap, n);
23     printf("\nArreglo 2:\n\n");
24     imprime(bp, n);
25     printf("\nArreglo 3:\n\n");
26     imprime(cp, m);
27
28     return 0;
29 }
30
31 void llena(int *p){
32     int i;
33     for(i=0; i<n; i++){
34         *(p+i)=rand()%n;
35     }
36 }
37
38 void imprime(int *p, int ne){
39     int i;
40     for(i=0; i<ne; i++){
41         printf(" %d ", *(p+i));
42         printf("\n");
43     }
44 }
45
46 void llenaC(int *a, int *b, int *c){
47     int i;
48     for(i=0; i<n; i++){
49         *c=*a;
50         a++;
51         *c=*b;
52         b++;
53         c++;
54     }

```

```
C:\Users\andik\Desktop\Actividad 3.exe
Arreglo 1:
1 7 4 0 9 4 8 8 2 4
Arreglo 2:
5 5 1 7 1 1 5 2 7 6
Arreglo 3:
1 5 7 5 4 1 0 7 9 1 4 1 8 5 8 2 2 7 4 6
-----
Process exited after 0.1447 seconds with return value 0
Presione una tecla para continuar . . .
```

Actividad 4

Escribir un programa en ANSI C donde se definan una función que calcule la longitud de una cadena de caracteres (arreglo de caracteres), considere que el fin de cadena está dado por el carácter '\0'.

```
1  #include <stdio.h>
2
3  void longCadena(char *ap, int *n);
4
5  int main(){
6      char cadena[30];
7      int nc;
8
9      printf("Introducir cadena: ");
10     //scanf("%s", cadena);
11     gets(cadena);
12
13     longCadena(cadena, &nc);
14
15     printf("longitud = %d", nc);
16     return 0;
17 }
18 char *gets(char *str);
19
20 void longCadena(char *ap, int *n){
21     int res=0;
22     *n=0;
23     while(*ap != '\0'){
24         res+=1;
25         ap+=1;
26     }
27     *n=res;
28 }
```

```
C:\Users\andik\Desktop\Actividad 4.exe
Introducir cadena: Hola mundo!!! :D
longitud = 16
-----
Process exited after 6.139 seconds with return value 0
Presione una tecla para continuar . . . _
```

Actividad 5

Realizar una suma de dos arreglos bidimensionales de 8X8 (matrices) utilizando apuntadores variables. Se debe tener una función `llena()` para llenar A y B y una función `suma()` que realice la suma de matrices $C=A+B$ y seguir la siguiente estructura del `main()`.

```
#include<stdio.h>
#define n 9
/*prototipo funciones*/
main(){
float A[n][n], B[n][n], C[n][n], *ap,*bp, *cp;
ap=
bp=
llena(ap);
llena(bp);
suma(ap,bp,cp);
    imprime(A);
    imprime(B);
    imprime(C);
}
/*Definición de funciones */
```



```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #define n 8
4
5  void llena (float *p);
6  void suma (float *a, float *b, float *c);
7  void imprime (float *p, char C);
8
9  int main (){
10     float A[n][n], B[n][n], C[n][n], *ap, *bp, *cp;
11     ap = &A[0][0];
12     bp = &B[0][0];
13     cp = &C[0][0];
14
15     llena (ap);
16     llena (bp);
17
18     suma (ap, bp, cp);
19
20     imprime (ap, 'A');
21     imprime (bp, 'B');
22     imprime (cp, 'C');
23
24     return 0;
25 }
26
27 void
28 llena (float *p)
29 {
30     int i;
31     for (i = 0; i < n * n; i++)
32     {
33         *(p + i) = rand () % n;
34     }
35 }

```

```

36
37 void suma (float *a, float *b, float *c)
38 {
39     int i;
40     for (i = 0; i < n * n; i++)
41     { *(c + i) = *(a + i) + *(b + i); }
42 }
43
44 void imprime (float *p, char c)
45 {
46     int i, j;
47     printf("\nMatriz %c\n\n", c);
48     for (i = 0; i < n; i++)
49     {
50         for (j = 0; j < n; j++){
51             printf (" %.1f", *(p + n * i + j));
52         }
53         printf ("\n");
54     }
55     printf("\n");
56 }

```

```
C:\Users\andik\Desktop\Actividad 5.exe

Matriz A

1.0  3.0  6.0  4.0  1.0  4.0  6.0  6.0
2.0  0.0  1.0  1.0  1.0  3.0  1.0  3.0
3.0  6.0  3.0  4.0  7.0  4.0  6.0  1.0
4.0  6.0  5.0  4.0  6.0  7.0  7.0  6.0
3.0  2.0  5.0  0.0  3.0  3.0  3.0  6.0
7.0  3.0  2.0  5.0  1.0  0.0  5.0  7.0
5.0  4.0  3.0  4.0  6.0  5.0  5.0  3.0
3.0  5.0  1.0  2.0  4.0  3.0  6.0  2.0

Matriz B

0.0  2.0  0.0  6.0  0.0  0.0  6.0  5.0
2.0  1.0  2.0  6.0  6.0  5.0  1.0  4.0
5.0  7.0  4.0  2.0  4.0  0.0  6.0  0.0
3.0  4.0  0.0  7.0  2.0  3.0  1.0  2.0
6.0  2.0  1.0  5.0  1.0  3.0  7.0  2.0
0.0  2.0  1.0  2.0  1.0  2.0  5.0  1.0
4.0  0.0  6.0  5.0  1.0  5.0  1.0  0.0
2.0  2.0  1.0  4.0  3.0  7.0  7.0  6.0
```

```
Matriz C

1.0  5.0  6.0  10.0  1.0  4.0  12.0  11.0
4.0  1.0  3.0  7.0  7.0  8.0  2.0  7.0
8.0  13.0  7.0  6.0  11.0  4.0  12.0  1.0
7.0  10.0  5.0  11.0  8.0  10.0  8.0  8.0
9.0  4.0  6.0  5.0  4.0  6.0  10.0  8.0
7.0  5.0  3.0  7.0  2.0  2.0  10.0  8.0
9.0  4.0  9.0  9.0  7.0  10.0  6.0  3.0
5.0  7.0  2.0  6.0  7.0  10.0  13.0  8.0

-----
Process exited after 0.1618 seconds with return value 0
Presione una tecla para continuar . . .
```

Práctica 3

Tipo de dato abstracto

Objetivo

Utilizar estructuras en lenguaje C para modelar un tipo de dato abstracto e implementarlo en las estructuras de datos lineales.

Desarrollo

Ejercicio 2

Una escuela necesita almacenar información de estudiantes en un arreglo unidimensional donde en cada elemento del mismo se almacene información sobre el número de cuenta e información del estudiante como su nombre, edad, sexo, edad y un arreglo unidimensional de 5 elementos correspondiente a sus notas. Se requiere realizar un programa con las siguientes definiciones de estructura para almacenar información de estudiantes utilizando un arreglo de estructuras estático.

```
struct DatosEstudiante{  
    char nombre[30];  
    Int edad;  
    char sexo;  
    int notas[5];  
};
```

```
struct registroEstudiante{  
    int numeroCuenta;
```

```
struct DatosEstudiante estudiante;  
};
```

El programa debe tener una función que permita registrar información de 3 tres estudiantes (desde teclado) y otra función que permita mostrar la información de todos los estudiantes.

```

1  #include <stdio.h>
2  #define c 3
3  #define v 5
4  struct dE{
5      char nombre[30];
6      int edad;
7      char sexo[1];
8      int notas[v]; };
9
10 struct rE{
11     int numeroCuenta;
12     struct dE estudiante;
13 };
14
15 void llenaDatos(struct rE *alum);
16 void imprimeDatos(struct rE *alum);
17
18 int main(){
19     struct rE alumnos[c];
20     llenaDatos(alumnos);
21     imprimeDatos(alumnos);
22     return 0; }
23
24 void llenaDatos(struct rE *alum){
25     int i,j;
26     for(i=0;i<c;i++){
27         printf("Ingrese los datos del alumno %i: \n",i+1);
28         printf("Nombre:");
29         scanf("%s", alum[i].estudiante.nombre);
30         printf("Numero de cuenta: ");
31         scanf("%d", &alum[i].numeroCuenta);
32         printf("Edad: ");
33         scanf("%d", &alum[i].estudiante.edad);
34         printf("Sexo[M/F]: ");
35         scanf("%s", alum[i].estudiante.sexo);

```

```

36         printf("\nIngrese las notas del alumno %d: \n", i+1);
37         for(j=0;j<v;j++){
38             printf("Nota %d: ", j+1);
39             scanf("%d" , &alum[i].estudiante.notas[j] ); }
40         printf("\n");}
41     }
42
43 void imprimeDatos(struct rE *alum){
44     int i,j;
45     for(i=0;i<c;i++){
46         printf("\n\nLos datos del alumno %i:\n",i+1);
47         printf("Nombre: %s\t", alum[i].estudiante.nombre);
48         printf("Numero de cuenta: %d\n",alum[i].numeroCuenta);
49         printf("Edad: %d \t \t", alum[i].estudiante.edad);
50         printf("Sexo:%s\n", alum[i].estudiante.sexo);
51         for(j=0;j<v;j++){
52             printf("Nota %d: %d    ", j+1, alum[i].estudiante.notas[j] ); }
53         }
54     }
55

```

C:\Users\andik\Desktop\Ejercicio 2.exe

Ingrese los datos del alumno 1:

Nombre:Jose

Numero de cuenta: 319254724

Edad: 19

Sexo[M/F]: M

Ingrese las notas del alumno 1:

Nota 1: 10

Nota 2: 9

Nota 3: 8

Nota 4: 9

Nota 5: 7

Ingrese los datos del alumno 2:

Nombre:Angel

Numero de cuenta: 319266743

Edad: 18

Sexo[M/F]: M

Ingrese las notas del alumno 2:

Nota 1: 10

Nota 2: 8

Nota 3: 9

Nota 4: 7

Nota 5: 6

Ingrese los datos del alumno 3:

Nombre:Pamela

Numero de cuenta: 319288745

Edad: 20

Sexo[M/F]: F

Ingrese las notas del alumno 3:

Nota 1: 10

Nota 2: 10

Nota 3: 10

Nota 4: 9

Nota 5: 8

Los datos del alumno 1:

Nombre: Jose Numero de cuenta: 319254724

Edad: 19 Sexo:M

Nota 1: 10 Nota 2: 9 Nota 3: 8 Nota 4: 9 Nota 5: 7

Los datos del alumno 2:

Nombre: Angel Numero de cuenta: 319266743

Edad: 18 Sexo:M

Nota 1: 10 Nota 2: 8 Nota 3: 9 Nota 4: 7 Nota 5: 6

Los datos del alumno 3:

Nombre: Pamela Numero de cuenta: 319288745

Edad: 20 Sexo:F

Nota 1: 10 Nota 2: 10 Nota 3: 10 Nota 4: 9 Nota 5: 8

Process exited after 54.04 seconds with return value 0

Presione una tecla para continuar

Ejercicio 3 (Actividad reporte)

Agregar al programa anterior una función que calcule e imprima cuál es el estudiante con mayor promedio en sus notas.

```
1  #include <stdio.h>
2  #define c 3
3  #define v 5
4  struct dE{
5      char nombre[30];
6      int edad;
7      char sexo[1];
8      int notas[v]; };
9
10 struct rE{
11     int numeroCuenta;
12     struct dE estudiante;
13 };
14
15 void llenaDatos(struct rE *alum);
16 void imprimeDatos(struct rE *alum);
17 void mejorEstudiante(struct rE *alum);
18
19 int main(){
20     struct rE alumnos[c];
21     llenaDatos(alumnos);
22     imprimeDatos(alumnos);
23     mejorEstudiante(alumnos);
24     return 0; }
25
26 void llenaDatos(struct rE *alum){
27     int i,j;
28     for(i=0;i<c;i++){
29         printf("Ingrese los datos del alumno %i:\n",i+1);
30         printf("Nombre:");
31         scanf("%s", alum[i].estudiante.nombre);
32         printf("Numero de cuenta:");
33         scanf("%d", &alum[i].numeroCuenta);
34         printf("Edad:");
35         scanf("%d", &alum[i].estudiante.edad);
```

```

40 printf("Nota %d: ", j+1);
41 scanf("%d" , &alum[i].estudiante.notas[j] ); }
42 printf("\n");}
43 }
44
45 void imprimeDatos(struct rE *alum){
46     int i,j;
47     for(i=0;i<c;i++){
48         printf("\nLos datos del alumno %i:\n",i+1);
49         printf("Nombre:%s\t", alum[i].estudiante.nombre);
50         printf("Numero de cuenta:%d\n",alum[i].numeroCuenta);
51         printf("Edad:%d \t \t", alum[i].estudiante.edad);
52         printf("Sexo:%s\n", alum[i].estudiante.sexo);
53         for(j=0;j<v;j++){
54             printf("Nota %d: %d", j+1, alum[i].estudiante.notas[j] ); }}}
55
56 void mejorEstudiante(struct rE *alumn){
57     printf("\n MEJOR ESTUDIANTE\n");
58     int indice_Mejor_alumn=-1;
59     float alumn_mejorPromedio=0;
60     int i,j;
61     for(i=0;i<3;i++){
62         float promedioEstudiante=0;
63         for(j=0;j<5;j++){
64             promedioEstudiante +=alumn[i].estudiante.notas[j];
65         }
66         promedioEstudiante=promedioEstudiante/5;
67         if(promedioEstudiante>alumn_mejorPromedio){
68             alumn_mejorPromedio=promedioEstudiante;
69             indice_Mejor_alumn=i;
70         }
71     }
72     printf("\n Estudiante con mejor promedio: %s",alumn[indice_Mejor_alumn].estudiante.nombre);
73     printf("\n Promedio del mejor estudiante: %.2f", alumn_mejorPromedio);
74 }

```


C:\Users\andik\Downloads\Ejercicio 3.exe

Ingrese los datos del alumno 1:
Nombre:Jose
Numero de cuenta:31827726
Edad:19
Sexo[M/F]:M

Ingrese las notas del alumno 1:
Nota 1: 10
Nota 2: 9
Nota 3: 9
Nota 4: 9
Nota 5: 9

Ingrese los datos del alumno 2:
Nombre:Juan
Numero de cuenta:319254724
Edad:18
Sexo[M/F]:M

Ingrese las notas del alumno 2:
Nota 1: 9
Nota 2: 10
Nota 3: 5
Nota 4: 7
Nota 5: 8

Ingrese los datos del alumno 3:
Nombre:Pamela
Numero de cuenta:11223343
Edad:20

C:\Users\andik\Downloads\Ejercicio 3.exe

Sexo[M/F]:F

Ingrese las notas del alumno 3:
Nota 1: 10
Nota 2: 9
Nota 3: 9
Nota 4: 8
Nota 5: 9

Los datos del alumno 1:
Nombre:Jose Numero de cuenta:31827726
Edad:19 Sexo:M
Nota 1: 10Nota 2: 9Nota 3: 9Nota 4: 9Nota 5: 9
Los datos del alumno 2:
Nombre:Juan Numero de cuenta:319254724
Edad:18 Sexo:M
Nota 1: 9Nota 2: 10Nota 3: 5Nota 4: 7Nota 5: 8
Los datos del alumno 3:
Nombre:Pamela Numero de cuenta:11223343
Edad:20 Sexo:F
Nota 1: 10Nota 2: 9Nota 3: 9Nota 4: 8Nota 5: 9
MEJOR ESTUDIANTE

Estudiante con mejor promedio: Jose
Promedio del mejor estudiante: 9.20

Process exited after 58.73 seconds with return value 0
Presione una tecla para continuar . . .

Ejercicio 4 (Actividad reporte)

Diseñar e implementar un programa que contenga una función que permitan multiplicar todos los elementos de números complejos en su forma exponencial almacenados en un arreglo unidimensional de 7 elementos. Se requiere se utilice la definición de una estructura para el manejo de números complejos.

```
1  #include<stdio.h>
2  #include<stdlib.h>
3  #include<math.h>
4  #define n 7
5  #define pi 3.1416
6
7  struct complejos{
8
9      double a;
10     double b;
11
12 };
13
14 int comex(struct complejos); //Funciones prototipo
15 double multis(struct complejos);
16
17 struct complejos K[n]; //Variables globales
18 double Mod[n], Expo[n];
19
20 int main(){
21
22     int i;
23
24     printf("\nArreglo de numeros complejos\n\n");
25
26     for(i=0; i<n; i++){ //Genero numeros aleatorios para el arreglo
27         K[i].a = rand()%12;
28         K[i].b = rand()%12;
29         printf("<1.01f> <1.01fi>\n", K[i].a, K[i].b); //Imprimo los numeros generados aleatoriamente del arreglo
30     }
31
32     printf("\nConvirtiendo numeros complejos a exponenciales\n\n");
33     printf("Modulos\t\tForma exponencial\n\n");
34
35     comex(K[n]);
36
37     printf("Multiplicando los complejos en su forma exponencial\n");
38
39     multis(K[n]);
40
41     return 0;
42 }
43
```

```

43
44 int comex(struct complejos l){ //Esta función convierte los numeros reales e imaginarios en modulo y forma exponencial respectivamente
45
46     int i;
47     double mod, argp, exp;
48
49     for(i=0; i<n; i++){
50
51         mod = pow(K[i].a, 2) + pow(K[i].b, 2);
52         mod = sqrt(mod);
53         argp = K[i].b;
54         argp = argp / K[i].a;
55         exp = atan(argp);
56         Mod[i] = mod; //Guardo los datos en otro arreglo para no influir en el original
57         Expo[i] = exp;
58
59         printf("<%1.2lf>\t\t<%1.2lf>\n\n", mod, exp);
60     }
61 }
62
63
64
65 double multis(struct complejos l){ // Multiplica los modulos y suma los exponentes
66
67     int i;
68     double res, resm = 1;
69
70     for(i=0; i<n; i++){
71
72         resm = Mod[i]*resm; //Cuando se multiplican numeros complejos en forma exponencial, solo el módulo se multiplica
73         res = Expo[i]+res; //Cuando se multiplican numeros complejos en su forma exponencial, su exponente se suma
74     }
75
76     printf("\nLa multiplicación de todos los modulos es: %lf", resm);
77     printf("\n\nLa suma de todas las formas exponenciales es: %lf", res);
78
79 }
80

```

```
C:\Users\andik\Desktop\Ejercicio 4.exe

Arreglo de numeros complejos

<5> <11i>
<10> <4i>
<5> <4i>
<6> <6i>
<10> <8i>
<5> <5i>
<1> <3i>

Convirtiendo numeros complejos a exponenciales

Modulos          Forma exponencial

<12.08>           <1.14>
<10.77>           <0.38>
<6.40>            <0.67>
<8.49>            <0.79>
<12.81>           <0.67>
<7.07>            <0.79>
<3.16>            <1.25>

Multiplicando los complejos en su forma exponencial

La multiplicaci%
n de todos los modulos es: 2024745.886278

La suma de todas las formas exponenciales es: 5.693999
-----
Process exited after 0.1263 seconds with return value 0
Presione una tecla para continuar . . .
```

Conclusiones

Ramírez Jiménez Aram:

- 1.- Consideró que el desarrollo de esta práctica nos induce a una mejor comprensión de los conocimientos teóricos ya vistos en clase, a su vez, nos muestra algunas aplicaciones del manejo de memoria y los apuntadores.
- 2.- Pudimos desarrollar un claro ejemplo de la aplicación de manejo de variables a través del uso de estructuras.

Romero Pizano Christian Gustavo:

- 1.- El desarrollo de esta práctica fue de suma importancia para comprender de mejor forma que son los apuntadores, además de su utilidad para el manejo de memoria en los programas que realizamos utilizando funciones, y lo más importante que es la forma de aplicarse estos apuntadores en el lenguaje C. El objetivo de la práctica y las actividades se cumplió de manera exitosa dado a que las instrucciones fueron claras y precisas.
- 2.- Durante esta práctica analizamos la forma y el uso de las estructuras en C, además de que se logró apreciar su capacidad de facilitar el manejo de variables en los programas que desarrollamos. Cumplimos con los objetivos de la práctica puesto que las actividades fueron desarrolladas con éxito.

Silverio Martínez Andrés:

- 1.- El objetivo de la práctica se cumplió, ya que con ayuda de apuntadores, se hizo más fácil la elaboración de los ejercicios propuestos en la práctica. Además, gracias a las clases con la profesora y el material que se nos ha sido de fácil acceso por parte de la misma, se pudo reforzar con éxito lo que son los apuntadores y cuál es su principal función. En el transcurso de la elaboración de la práctica no encontramos ningún inconveniente con ninguna de las actividades realizadas.
- 2.- El objetivo de la práctica se ha cumplido de manera exitosa, en toda la práctica se utilizaron estructuras de tipo abstracto (struct) para la facilitación al realizar todos los programas de la práctica. Al momento de estar programando, no surgió alguna duda de algún tipo, finalizando en una práctica exitosa.

Referencias

- Curso: Estructura de Datos Y Algoritmos I 2022-2 G04 (2022). Práctica2EDAI2021-2.pdf. Recuperado el 02 de marzo de 2022, de http://telematica1.fi-b.unam.mx/aula/main/document/showinframes.php?cidReq=EDI22IG04&id_session=0&qidReq=0&gradebook=0&origin=&id=794
- Laboratorio salas A y B (S.f). Carátula. Recuperado el 02 de marzo de 2022, de <http://lcp02.fi-b.unam.mx/>